



Automated Unpacking

Tool: https://github.com/hasherezade/mal_unpack

Executive Summary

The sample is **position-independent shellcode** (PIC) for 32-bit Windows. It implements **runtime API resolution by hash**, **decompression of an embedded payload**, **CPUID-based** environment checks**, and **thread-based execution** with a state machine. No import table is present; the only static string is 'KERNEL32.dll', indicating manual resolution of Windows APIs from kernel32. The code is consistent with **loader/dropper or **first-stage shellcode** commonly seen in exploit payloads, packed malware, or modular implants.

- **Risk level : High** — capable of loading and executing arbitrary second-stage payloads, evading static import analysis, and adapting execution based on CPU environment.

File Properties

Property	Value
File Name	mount.exe
MD5	5597dc44aecdb621cb115c3157b8b283
SHA-1	8b70fcc08fc3125f1a141f75ebfc9762ddb133b0
SHA-256	ac2309dcfaaf6e95c5de49910db3c8f0c44ce36948ad5fa61320cab0315b0b5b
Vhash	015046656d151az45nz1ez1
Authentihash	b45403c22c16b911f07ccd89488f193bfc2baa61af5267cea0738112d2cedbc5
Imphash	e28e04a7ac948b435bd640e83b2d285c
Rich PE Header Hash	05e00c445df281f24266aee892fddb93
SSDEEP	3072:4efx+Z+Fj0D/aPkInA/n7k17m56mztCb+ZLhzISalJ/5ISf:4e4+pKAA/gJm5tg+ZLhzialjyu
TLSH	T1C904C0223A40D473D427A1724860DA74EF3866714B7C950B7BE903BE9F603E0923E35A
File Type	Win32 EXE (executable, windows, pe, peexe)
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
TrID	• Win32 Executable MS Visual C++ (47.3%) • Win64 Executable (15.9%) • Win32 Dynamic Link Library (9.9%) • Win16 NE executable (7.6%) • Win32 Executable (6.8%)
DetectItEasy	• PE32 • Compiler: Microsoft Visual C/C++ (2008-2010) • Linker: Microsoft Linker (9.00.21022) • Tool: Visual Studio (2008)
Magika	PEBIN
File Size	174.50 KB (178,688 bytes)

PE Sections

Property	.text (Section 0)	.rdata (Section 1)	.data (Section 2)	.rsrc (Section 3)
Entropy	6.723	6.279	3.513	4.072
SHA-256	7DEA6528B5B...	D57077C9F62...	4B6A56676EF...	43C67A2F650...
File Ratio	40.11%	55.30%	3.44%	0.57%
Raw Address (Start)	0x00000400	0x00011C00	0x00029E00	0x0002B600
Raw Address (End)	0x00011C00	0x00029E00	0x0002B600	0x0002BA00

Raw Size	71,680 bytes	98,816 bytes	6,144 bytes	1,024 bytes
Virtual Address	0x00001000	0x00013000	0x0002C000	0x00030000
Virtual Size	71,471 bytes	98,306 bytes	13,148 bytes	874 bytes

Observations

- **Entropy:** The `.text` section (where the executable code lives) has an entropy of **6.723**. While high, it is generally below the typical threshold for packed malware (usually >7.0), suggesting the code is likely obfuscated or compressed but perhaps not fully packed with a tool like UPX.
- **Virtual vs. Raw Size:** In the `.data` section, the virtual size (13,148 bytes) is significantly larger than the raw size (6,144 bytes). This is common for uninitialized data (BSS) that occupies space in memory but not on disk.

IAT

Function Name	Flag	Binding	Extra	Address 1	Address 2	Library
WriteFile	x	implicit	-	0x0002AC8A	0x0002AC8A	KERNEL32.dll
WriteConsoleW	-	implicit	-	0x0002AF8E	0x0002AF8E	KERNEL32.dll
WriteConsoleA	-	implicit	-	0x0002AF68	0x0002AF68	KERNEL32.dll
WideCharToMultiByte	-	implicit	-	0x0002ADFC	0x0002ADFC	KERNEL32.dll
VirtualFree	-	implicit	-	0x0002AC50	0x0002AC50	KERNEL32.dll
VirtualAlloc	x	implicit	-	0x0002AC5E	0x0002AC5E	KERNEL32.dll
UnhandledExceptionFilter	-	implicit	-	0x0002AB42	0x0002AB42	KERNEL32.dll
TlsSetValue	-	implicit	-	0x0002AD2A	0x0002AD2A	KERNEL32.dll
TlsGetValue	-	implicit	-	0x0002AD10	0x0002AD10	KERNEL32.dll
TlsFree	-	implicit	-	0x0002AD38	0x0002AD38	KERNEL32.dll
TlsAlloc	-	implicit	-	0x0002AD1E	0x0002AD1E	KERNEL32.dll
TerminateProcess	-	implicit	-	0x0002AB1A	0x0002AB1A	KERNEL32.dll
Sleep	-	implicit	-	0x0002AACE	0x0002AACE	KERNEL32.dll
SetUnhandledExceptionFilter	-	implicit	-	0x0002AB5E	0x0002AB5E	KERNEL32.dll
SetStdHandle	-	implicit	-	0x0002AEAA	0x0002AEAA	KERNEL32.dll
SetLastError	-	implicit	-	0x0002AD42	0x0002AD42	KERNEL32.dll
SetHandleCount	-	implicit	-	0x0002ABC0	0x0002ABC0	KERNEL32.dll
SetFilePointer	-	implicit	-	0x0002AF56	0x0002AF56	KERNEL32.dll
RtlUnwind	-	implicit	-	0x0002AC08	0x0002AC08	KERNEL32.dll
RaiseException	x	implicit	-	0x0002AC3E	0x0002AC3E	KERNEL32.dll
QueryPerformanceCounter	-	implicit	-	0x0002AE2C	0x0002AE2C	KERNEL32.dll
MultiByteToWideChar	-	implicit	-	0x0002AE86	0x0002AE86	KERNEL32.dll
LoadStringW	-	implicit	-	0x0002AA94	0x0002AA94	USER32.dll
LoadLibraryA	-	implicit	-	0x0002AD78	0x0002AD78	KERNEL32.dll
LeaveCriticalSection	-	implicit	-	0x0002ABA8	0x0002ABA8	KERNEL32.dll
LCMapStringW	-	implicit	-	0x0002AD68	0x0002AD68	KERNEL32.dll
LCMapStringA	-	implicit	-	0x0002AEFC	0x0002AEFC	KERNEL32.dll
IsValidCodePage	-	implicit	-	0x0002ACFE	0x0002ACFE	KERNEL32.dll
IsDebuggerPresent	-	implicit	-	0x0002AB7C	0x0002AB7C	KERNEL32.dll
InterlockedIncrement	-	implicit	-	0x0002ACB8	0x0002ACB8	KERNEL32.dll
InterlockedDecrement	-	implicit	-	0x0002ACD0	0x0002ACD0	KERNEL32.dll
InitializeCriticalSectionAndSpinCount	-	implicit	-	0x0002AD88	0x0002AD88	KERNEL32.dll
HeapSize	-	implicit	-	0x0002AEF0	0x0002AEF0	KERNEL32.dll
HeapReAlloc	-	implicit	-	0x0002AC6E	0x0002AC6E	KERNEL32.dll
HeapFree	-	implicit	-	0x0002AC24	0x0002AC24	KERNEL32.dll

HeapCreate	-	implicit	-	0x0002AC7C	0x0002AC7C	KERNEL32.dll
HeapAlloc	-	implicit	-	0x0002AAAE	0x0002AAAE	KERNEL32.dll
GetTickCount	-	implicit	-	0x0002AE46	0x0002AE46	KERNEL32.dll
GetSystemTimeAsFileTime	-	implicit	-	0x0002AE6C	0x0002AE6C	KERNEL32.dll
GetStringTypeW	-	implicit	-	0x0002AF1E	0x0002AF1E	KERNEL32.dll
GetStringTypeA	-	implicit	-	0x0002AF0C	0x0002AF0C	KERNEL32.dll
GetStdHandle	-	implicit	-	0x0002ABD2	0x0002ABD2	KERNEL32.dll
GetStartupInfoA	-	implicit	-	0x0002AB08	0x0002AB08	KERNEL32.dll
GetProcAddress	-	implicit	-	0x0002AAD6	0x0002AAD6	KERNEL32.dll
GetOEMCP	-	implicit	-	0x0002ACF2	0x0002ACF2	KERNEL32.dll
GetModuleHandleW	-	implicit	-	0x0002AABA	0x0002AABA	KERNEL32.dll
GetModuleHandleA	-	implicit	-	0x0002AF42	0x0002AF42	KERNEL32.dll
GetModuleFileNameA	-	implicit	-	0x0002AC96	0x0002AC96	KERNEL32.dll
GetLocaleInfoA	-	implicit	-	0x0002AF30	0x0002AF30	KERNEL32.dll
GetLastError	-	implicit	-	0x0002AC14	0x0002AC14	KERNEL32.dll
GetFileType	-	implicit	-	0x0002ABE2	0x0002ABE2	KERNEL32.dll
GetEnvironmentStringsW	x	implicit	-	0x0002AE12	0x0002AE12	KERNEL32.dll
GetEnvironmentStrings	x	implicit	-	0x0002ADCA	0x0002ADCA	KERNEL32.dll
GetCurrentThreadId	x	implicit	-	0x0002AD52	0x0002AD52	KERNEL32.dll
GetCurrentProcessId	x	implicit	-	0x0002AE56	0x0002AE56	KERNEL32.dll
GetCurrentProcess	x	implicit	-	0x0002AB2E	0x0002AB2E	KERNEL32.dll
GetConsoleOutputCP	-	implicit	-	0x0002AF78	0x0002AF78	KERNEL32.dll
GetConsoleMode	-	implicit	-	0x0002AECA	0x0002AECA	KERNEL32.dll
GetConsoleCP	-	implicit	-	0x0002AEBA	0x0002AEBA	KERNEL32.dll
GetCommandLineA	-	implicit	-	0x0002AAF6	0x0002AAF6	KERNEL32.dll
GetCPIInfo	-	implicit	-	0x0002ACAC	0x0002ACAC	KERNEL32.dll
GetACP	-	implicit	-	0x0002ACE8	0x0002ACE8	KERNEL32.dll
FreeEnvironmentStringsW	-	implicit	-	0x0002ADE2	0x0002ADE2	KERNEL32.dll
FreeEnvironmentStringsA	-	implicit	-	0x0002ADB0	0x0002ADB0	KERNEL32.dll
FlushFileBuffers	-	implicit	-	0x0002AEDC	0x0002AEDC	KERNEL32.dll
ExitProcess	-	implicit	-	0x0002AAE8	0x0002AAE8	KERNEL32.dll
EnterCriticalSection	-	implicit	-	0x0002AB90	0x0002AB90	KERNEL32.dll
DeleteCriticalSection	-	implicit	-	0x0002ABF0	0x0002ABF0	KERNEL32.dll
CreateFileA	-	implicit	-	0x0002AE9C	0x0002AE9C	KERNEL32.dll
CloseHandle	-	implicit	-	0x0002AC30	0x0002AC30	KERNEL32.dll

Description

The Import Address Table (IAT) is a key structure in PE files that lists the external functions the program needs to run.

- **Evasion Indicators:** The presence of `IsDebuggerPresent` suggests the file may have anti-debugging capabilities to detect if it's being analyzed.
- **Memory Manipulation:** Functions like `VirtualAlloc` and `VirtualFree` are often used by malware for shellcode injection or unpacking.
- **Discovery:** `GetModuleFileNameA` and `GetCommandLineA` are commonly used to gather environment data or verify its own location before executing further stages.

Automated unpacking using `mal_unpack`

```

FLARE-VM Thu 02/12/2026 9:59:32.09
C:\Users\redteam\Desktop>mal_unpack.exe /exe mount.exe /timeout 10000
[*] Cache is Disabled!
Starting the process: mount.exe
With cmdline: ""
Exe name: mount.exe
Root Dir: mount.exe.out
[*] Watch respawns from main EXE file: mount.exe
Module Path retrieved: C:\Users\redteam\Desktop\mount.exe
Scanning...
Found suspicious: 4396
Suspicious detected, breaking!
Unpacked in: 875 milliseconds; 1 attempts.
WARNING: 1 of the related processes are not killed

```

Fig 1 - Unpacking using mal_unpack

This tool unpacked the sample and put it in mount.exe.out directory.

Name	Date modified	Type	Size
56053f.exe	2/12/2026 9:59 AM	Application	59 KB
560000.shc	2/12/2026 9:59 AM	SHC File	60 KB
570000.exe	2/12/2026 9:59 AM	Application	85 KB
570000.exe.imports.txt	2/12/2026 9:59 AM	Text Document	5 KB
dump_report.json	2/12/2026 9:59 AM	JSON Source File	1 KB
scan_report.json	2/12/2026 9:59 AM	JSON Source File	2 KB

Fig 2 - Output dir

There's a lot of files we'll be focusing on `.shc` shellcode file.

Binary Ninja Analysis

Open the shellcode file in binary ninja. Choose appropriate platform `windows-x86`

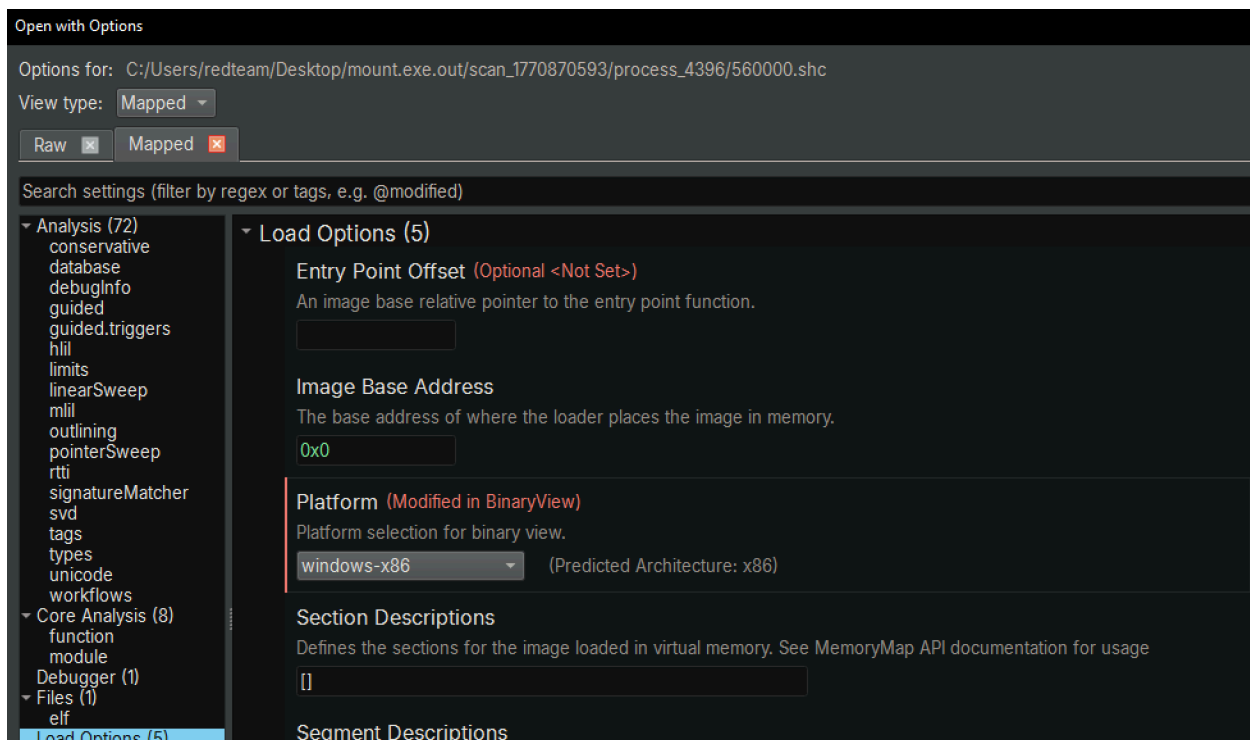


Fig 3 - Opening shellcode in binary ninja

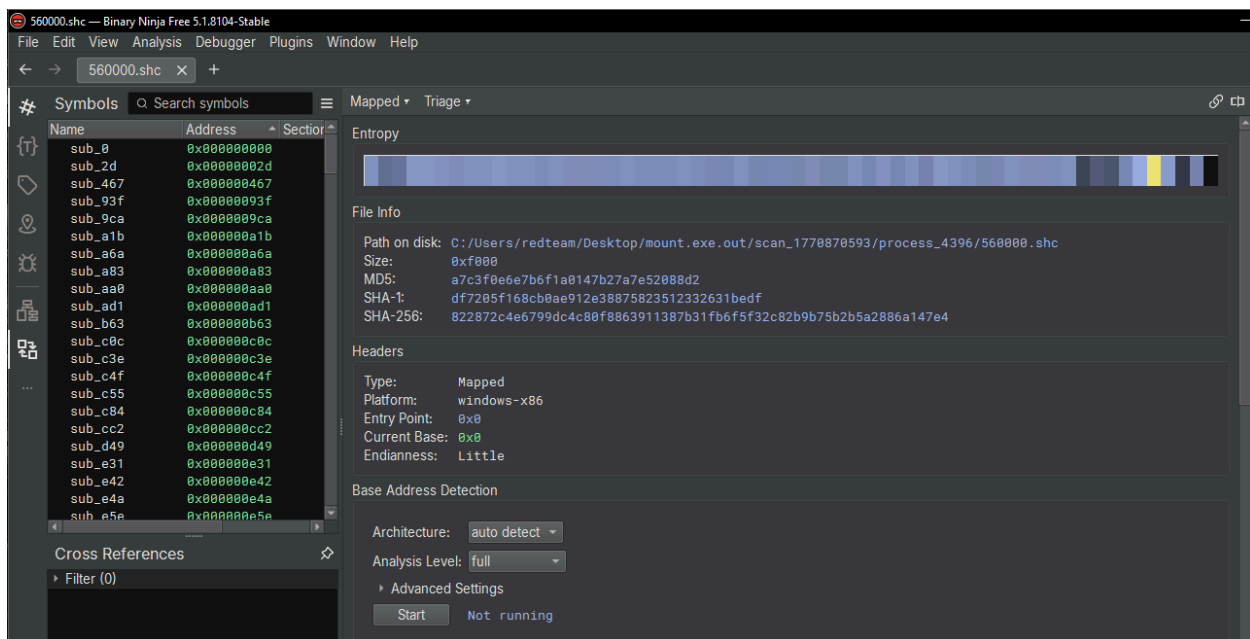


Fig 4 - binary ninja Triage summary

Switch to linear for high level representation.

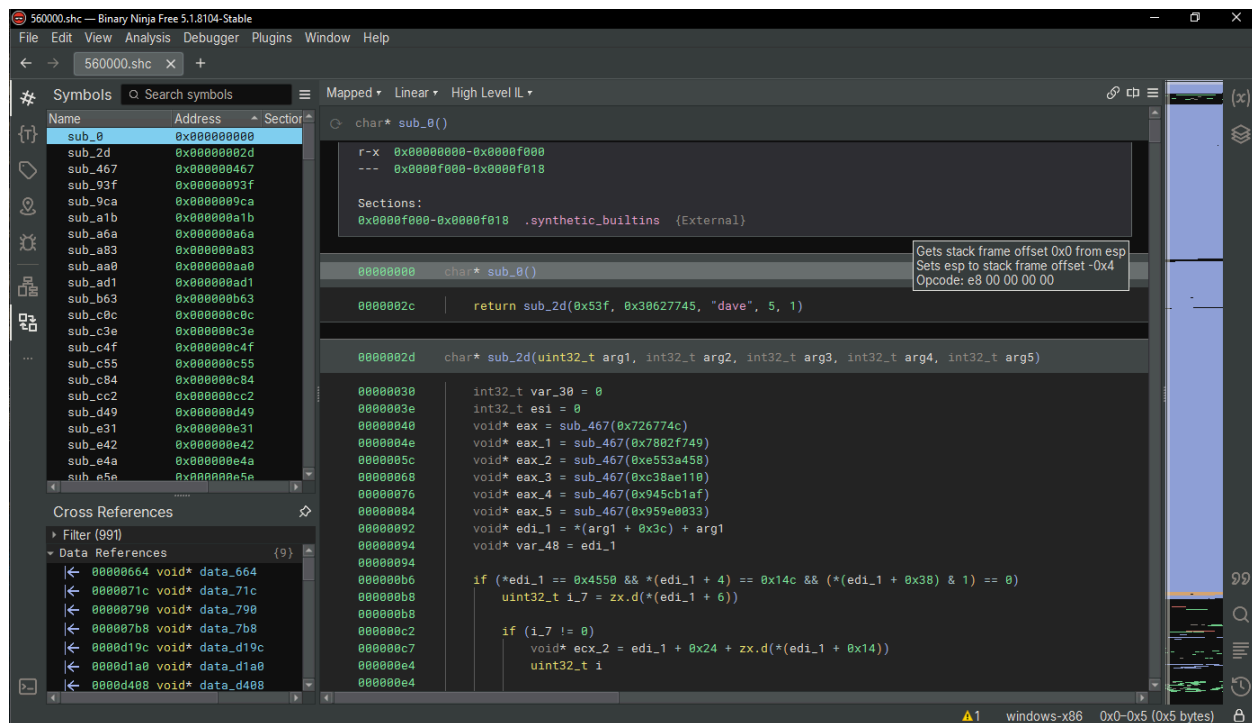


Fig 5 - Linear View of binary ninja

The sample utilizes a custom implementation of **dynamic API resolution** to hide its true capabilities from static analysis tools and the Import Address Table (IAT).

Mechanism: PEB Traversing & ROR13 Hashing

Instead of calling Windows APIs directly, the malware manually locates the **Process Environment Block (PEB)** via the Thread Environment Block (TEB) at `fs:[30h]`. It then iterates through the `InLoadOrderModuleList` to find loaded system DLLs (typically `kernel32.dll` or `ntdll.dll`).

Once a DLL is located, the malware parses the **PE Export Directory** to find function names.

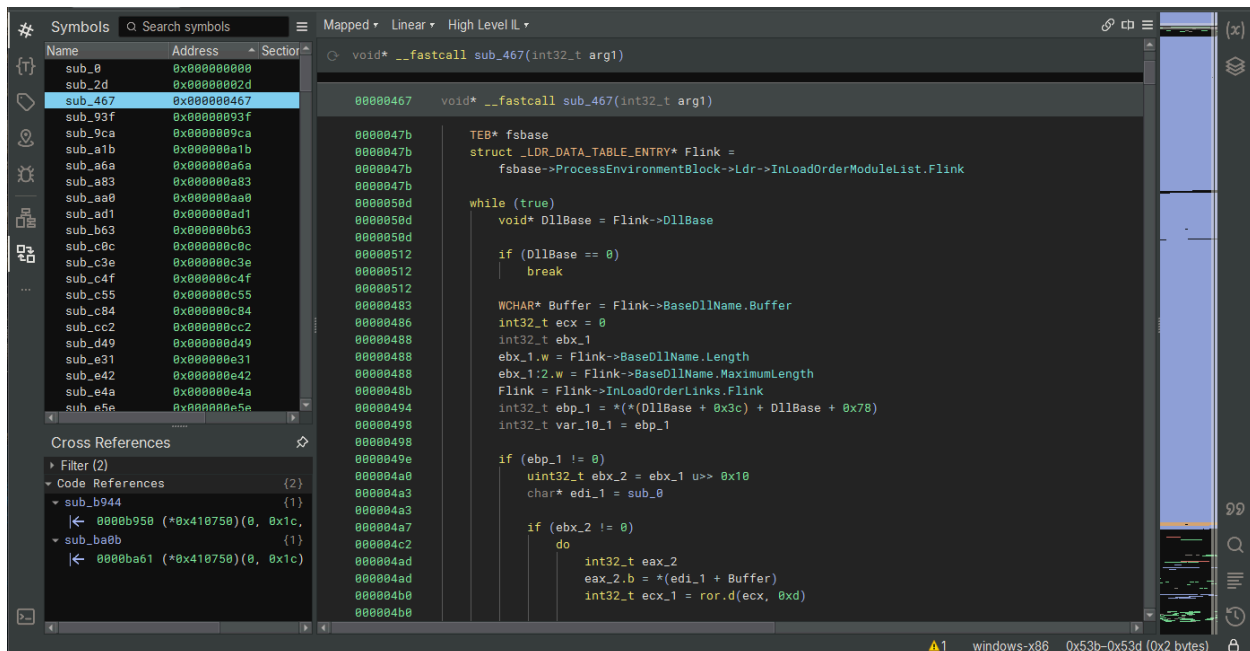


Fig 6 - Function `sub_467(0x726774c)`

Shellcode Information

Field	Value
Format	Raw shellcode (no PE/ELF header in analyzed dump)
Architecture	x86 (32-bit)
Size	~61 KB (0x00000000 – 0x0000EFFF)
Entry point	<code>shellcode_entry</code> (offset <code>0x0000B78D</code> in dump)
Static strings	<code>KERNEL32.dll</code> (offset <code>0x0000D45F</code>)
Imports	None (dynamic resolution only)
Exports	None

Technical Analysis

Memory layout

Component	Description
Single Segment	One contiguous executable/data region (<code>ram</code>) from base to <code>0xEFFF</code> .
API Table	Function pointers stored at offset <code>0x0000</code> . The first slot acts as an API trampoline for dynamic calls.
Hash Tables	API hash lists located at offsets <code>0xEF00</code> and <code>0xFFC0</code> , referenced during the initialization phase.

Anti-Analysis & Evasion

Technique	Implementation
No Import Table	All Windows APIs are resolved at runtime; static analysis cannot identify imported functions.
API Hashing	Export names are hashed using the formula: $\$hash = hash \setminus \text{times } 0x1003f + *name\$$.

Single Static String	Only <code>KERNEL32.dll</code> exists in plaintext; no other DLL or API names are present.
CPUID Checks	Verifies "GenuineIntel" and specific Intel family/model/stepping (e.g., <code>0x106C0</code> , <code>0x20660</code>). Likely used to detect and avoid VM/Sandbox environments.
Timing	Implements tick-count-based delays (3–8 second ranges) to bypass automated sandbox analysis.

API Resolution (PEB-based)

The shellcode utilizes a multi-step process to interact with the Windows OS without static linked dependencies:

1. **Kernel32 Base Location:** Obtained via the **PEB** (Process Environment Block) by traversing `PEB -> Ldr -> InMemoryOrderModuleList`.
2. **Export Parsing:** The shellcode manually parses the PE header to locate the **Export Directory** (RVA `0x78`).
3. **Hashing:** Each exported name is processed through the hashing algorithm at offset `0x0000A6A`.
4. **Table Population:** Resolved addresses are written into the API table at `0x40D000`. The specific APIs selected are driven by hash lists at `0x40EF00` and `0x40FFC0`.
5. **Invocation via Trampoline:** Execution is passed through an **API trampoline** at `0x0000C7B1` (`JMP [0x40D000]`), ensuring no direct API references exist at the actual call sites.

Execution flow

```

shellcode_entry (0xB78D)
├─ Resolve kernel32 (PEB)
├─ Get system directory / build path
├─ CreateThread(..., FUN_0000A524, ...)
├─ FUN_0000A524 (message/wait loop)
│   └─ Wait → FUN_0000A465 (state machine)
│       State 0: FUN_00005C69, FUN_00006B55, FUN_0000C04A (init, resolve APIs, create thread)
│       State 1: FUN_00007EF1 ... FUN_00009A2F, FUN_00005A04 (setup)
│       State 2: FUN_0000A358 (main logic)
│           ├─ GetTickCount, timing
│           ├─ FUN_0000A5D7
│           ├─ FUN_00005A43 (decompression)
│           │   └─ Inflate (zlib-style) → decompressed payload
│           ├─ FUN_00005BD6 / FUN_0000C0FC / FUN_0000A679 (payload execution path)
│           └─ Optional: FUN_0000C384, FUN_0000C0B7, FUN_0000C45D
│       State 3: Cleanup (e.g. close handle)

```

Payload Handling

Property	Details
Algorithm	Zlib-compatible Deflate/Inflate.
Core Routine	<code>FUN_00004413</code> (Inflate implementation).
Data Flow	<code>FUN_00005812</code> → <code>FUN_000023B2</code> → <code>FUN_0000218E</code> .
Buffer Strategy	Uses an 8 KB ring buffer to maintain stream state during decompression.
Dual-Capability	<code>FUN_000028C1</code> builds Huffman/length tables for Deflate, suggesting the malware can also compress data (potentially for exfiltration or C2 staging).

Execution Logic: Routine `FUN_00005A43` initializes the inflate pipeline. The resulting decompressed data is then passed to `FUN_0000A679`, which likely handles the execution of the next-stage binary or the parsing of a decrypted configuration file.

Environment & Capability Detection

The sample performs deep hardware inspection to ensure it is running on a specific target environment and to avoid detection by security researchers using generic virtual machines.

- **CPUID Validation:** The malware executes the `CPUID` instruction to confirm the "GenuineIntel" vendor string.
- **Signature Matching:** It compares specific Intel family/model/stepping signatures against internal globals (`_DAT_00411AD8`).
- **Feature Bit Inspection:** Uses **Extended CPUID (Leaf 7)** to check for specific hardware features (bits `0x200` and `0x20`).
- **Environment Flags:** Based on these checks, it assigns a "capability level" (0–5). This likely dictates which malicious features are activated or whether the malware terminates to avoid a sandbox.

Inferred Windows API Usage

Since the malware uses dynamic resolution via ROR13 hashing, the following APIs are inferred based on the operational context of the code and observed memory constants:

Inferred Role	Usage Context
<code>GetProcAddress</code> / <code>LoadLibrary</code>	Core resolution of <code>kernel32.dll</code> exports (e.g., trampoline index 10).
<code>GetTickCount</code>	Implements anti-sandbox timing and execution pacing (3–8s delays).
<code>VirtualAlloc</code> / <code>HeapAlloc</code>	Memory management for the decompressed payload (allocator at <code>0x40FD8C</code>).
<code>CreateThread</code>	Asynchronous execution; spawns a worker thread for the main loop (<code>FUN_0000A524</code>).
<code>WaitForSingleObject</code>	Event synchronization and loop timing (4–8s intervals).
<code>GetSystemDirectory</code>	Environment discovery; likely used to locate target paths for dropped files.
<code>memcpy</code> / <code>memset</code>	Native memory manipulation for block copying and zeroing out state.

Behavioral Summary

The following table summarizes the execution lifecycle of the shellcode from initial entry to final payload delivery.

Phase	Behavior
1. Load	Shellcode is injected or executed via an exploit, packed stub, or reflective loader.
2. Init	Manually traverses the PEB to locate <code>kernel32.dll</code> ; resolves APIs by hash and populates the jump table.
3. Setup	Hardware inspection (CPUID) is performed; memory is allocated; environment paths are gathered.
4. Run	Enters a timed wait loop. Once sandbox/timing checks pass, the embedded payload is decompressed (Zlib) and executed in memory.
5. Cleanup	Reaches "State 3": Closes open handles, clears sensitive memory regions, and terminates the process.

Indicators of Compromise (IOCs)

Static Indicators

Type	Value
String	<code>KERNEL32.dll</code>
Hash Constant	API hash multiplier <code>0x1003f</code>
Code Pattern	Indirect call via <code>JMP [0x40D000]</code> (or equivalent at runtime base)
CPUID Check	"GenuineIntel" (<code>0x49656E69</code> , <code>0x6C65746E</code> , <code>0x756E6547</code>)
Key Offsets	Entry: <code>0xB78D</code> ; Trampoline: <code>0xC7B1</code> ; Hash: <code>0xA6A</code> ; Resolver: <code>0xB63</code>

Capabilities Assessment

This assessment provides a high-level overview of the functions currently integrated into the analyzed shellcode.

Capability	Present	Notes
Runtime API Resolution	Yes	Hash-based; focuses on <code>kernel32.dll</code> in this specific sample.
Payload Decompression	Yes	Uses a Zlib-style inflate engine for secondary stage delivery.
Thread Creation	Yes	Main malicious logic is offloaded to a separate worker thread.
Environment Check	Yes	Rigorous Intel CPUID and feature flag verification.
Anti-Sandbox Timing	Yes	Uses <code>GetTickCount</code> to implement execution delays.
Persistence	No	Not observed; likely handled by the second-stage payload.
Network / C2	No	Not observed in this loader; likely resides in the decompressed data.
File / Registry Ops	No	Likely implemented in the next stage after API resolution.